Probar números aleatorios enteros

Intentos:

```
numeroUno = 65
numeroDos = 28

numeroUno = 27
numeroDos = 52

numeroUno = 6
numeroDos = 29

numeroUno = 80
numeroDos = 33

numeroUno = 90
numeroDos = 10

numeroUno = 78
numeroDos = 35

numeroUno = 0
numeroDos = 1

numeroUno = 73
numeroDos = 94

numeroUno = 92
numeroDos = 52

numeroUno = 76
numeroDos = 18
```

Probar números aleatorios enteros

Intentos:

```
numeroUno = 72
numeroDos = 50
numeroTres = 81
numeroCuatro = 71
numeroFloatUno = 72.809998
numeroFloatDos = 50.709999

numeroUno = 1
numeroDos = 17
numeroTres = 52
numeroCuatro = 67
numeroFloatUno = 1.520000
numeroFloatDos = 17.670000

numeroUno = 8
numeroDos = 65
numeroTres = 12
numeroCuatro = 84
numeroFloatUno = 8.120000
numeroFloatDos = 65.839996

numeroUno = 65
numeroDos = 42
numeroTres = 92
numeroCuatro = 82
numeroFloatUno = 65.919998
numeroFloatDos = 42.820000

numeroUno = 7
numeroDos = 55
numeroTres = 81
numeroCuatro = 36
numeroFloatUno = 7.810000
numeroFloatDos = 55.360001

numeroUno = 35
numeroDos = 43
numeroTres = 28
numeroCuatro = 33
numeroFloatUno = 35.279999
numeroFloatDos = 43.330002
```

```
numeroUno = 91
numeroDos = 98
numeroTres = 61
numeroCuatro = 85
numeroFloatUno = 91.610001
numeroFloatDos = 98.849998

numeroUno = 6
numeroDos = 90
numeroTres = 54
numeroCuatro = 22
numeroFloatUno = 6.540000
numeroFloatDos = 90.220001

numeroUno = 67
numeroDos = 22
numeroTres = 95
numeroCuatro = 92
numeroFloatUno = 67.949997
numeroFloatDos = 22.920000

numeroUno = 1
numeroDos = 89
numeroTres = 55
numeroCuatro = 38
numeroFloatUno = 1.550000
numeroFloatDos = 89.379997
```

Programa caracter aleatorio

Parte importante:

```
    // Genera letras minúsculas (ASCII 97 - 122)
    letraMinusculaUno = rand() % 26 + 97;
    letraMinusculaDos = rand() % 26 + 97;


    // Genera letras mayúsculas (ASCII 65 - 90)
    letraMayusculaUno = rand() % 26 + 65;
    letraMayusculaDos = rand() % 26 + 65;

    // Genera caracteres imprimibles (ASCII 33 - 126)
    caracterUno = rand() % 94 + 33;
    caracterDos = rand() % 94 + 33;
```

```
Letras minusculas:
letraMinusculaUno = f
letraMinusculaDos = x

Letras mayusculas:
letraMayusculaUno = D
letraMayusculaDos = O

Caracteres ASCII imprimibles:
caracterUno = l
caracterDos = ^
```

operadorconchar.c

Resultado:

```
===== TIPO: unsigned char =====
x =  38 ('&')
y = 241

== Aritmeticos (unsigned char) ==
x + y  = 279  -> unsigned char: 23
x - y  = -203  -> unsigned char: 53
x * y  = 9158  -> unsigned char: 198
x / y  = 0  -> unsigned char: 0 (division entera)
x % y  = 38  -> unsigned char: 38
+x     = 38  -> unsigned char: 38
-x     = -38  -> unsigned char: 218

== Incremento/Decremento ==
t inicial = 38
t++  imprime 38, luego t=38
++t  ahora t=40
t--  imprime 40, luego t=40
--t  ahora t=38

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
(x && y)  -> 1
```

```
(x || y)  -> 1

== Bits ==
~x        = 4294967257
x & y     = 32
x | y     = 247
x ^ y     = 215
x << 1    = 76
x >> 1    = 19

== Asignacion ==
a = x =  38 ('&')
a += y =  23
a -= y =  38 ('&')
a *= y = 198
a /= y =   0
a %= y =   0
a <<= 1 =   0
a >>= 1 =   0
a &= y =   0
a ^= y = 241
a |= y = 241

== Ternario y sizeof ==
mayor (x>y?x:y) = 241
sizeof(unsigned char) = 1 bytes
sizeof(x)             = 1 bytes

== Casting desde x ==
char        = 38
short       = 38
int         = 38
long        = 38
long long   = 38
float       = 38.000000
double      = 38.000000
```

```
long double = 38.000000
```

Desbordamiento (overflow y underflow)

El tipo `unsigned char` utiliza 8 bits, por lo que solo puede almacenar valores entre **0 y 255**.

Cuando una operación aritmética produce un valor mayor que 255 o menor que 0, ocurre un desbordamiento.

En tipos **unsigned**, el estándar de C define que el resultado se ajusta mediante **aritmética modular**:

resultado = valor mod 256

Por ejemplo:

- `38 + 241 = 279` → `279 mod 256 = 23` (overflow)
- `38 - 241 = -203` → `-203 mod 256 = 53` (underflow)

Esto demuestra que en enteros sin signo el desbordamiento no genera error, sino que el valor “se envuelve” dentro del rango permitido.

operadorconshort.c

Resultado:

```
===== TIPO: unsigned char =====
x =  38 ('&')
y = 241

== Aritmeticos (unsigned char) ==
x + y  = 279  -> unsigned char: 23
x - y  = -203  -> unsigned char: 53
x * y  = 9158  -> unsigned char: 198
x / y  = 0  -> unsigned char: 0 (division entera)
x % y  = 38  -> unsigned char: 38
+x     = 38  -> unsigned char: 38
-x     = -38  -> unsigned char: 218

== Incremento/Decremento ==
t inicial = 38
t++  imprime 38, luego t=38
++t  ahora t=40
t--  imprime 40, luego t=40
--t  ahora t=38

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
(x && y)  -> 1
(x || y)  -> 1
```

```
== Bits ==
~x        = 4294967257
x & y     = 32
x | y     = 247
x ^ y     = 215
x << 1    = 76
x >> 1    = 19

== Asignacion ==
a = x =  38 ('&')
a += y =  23
a -= y =  38 ('&')
a *= y = 198
a /= y =   0
a %= y =   0
a <<= 1 =   0
a >>= 1 =   0
a &= y =   0
a ^= y = 241
a |= y = 241

== Ternario y sizeof ==
mayor (x>y?x:y) = 241
sizeof(unsigned char) = 1 bytes
sizeof(x)             = 1 bytes

== Casting desde x ==
char        = 38
short       = 38
int         = 38
long        = 38
long long   = 38
float       = 38.000000
double      = 38.000000
long double = 38.000000
```

operadorconint.c

Resultado:

```
===== TIPO: int =====
x = 185

y = 233


== Aritmeticos (int) ==
x + y  = 418  -> int: 418
x - y  = -48  -> int: -48
x * y  = 43105  -> int: 43105
x / y  = 0  -> int: 0 (division entera)
x % y  = 185  -> int: 185
+x     = 185
-x     = -185

== Incremento/Decremento ==
t inicial = 185
t++  imprime 185, luego t=186
++t  ahora t=187
t--  imprime 187, luego t=186
--t  ahora t=185

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
```

```
(x && y)  -> 1
(x || y)  -> 1

== Bits ==
~x        = 4294967110
x & y     = 169
x | y     = 249
x ^ y     = 80
x << 1    = 370
x >> 1    = 92

== Asignacion ==
a = x = 185

a += y = 418

a -= y = 185

a *= y = 43105

a /= y = 185

a %= y = 185

a <<= 1 = 370

a >>= 1 = 185

a &= y = 169

a ^= y = 64
 ('@')
a |= y = 233


== Ternario y sizeof ==
```

```
mayor (x>y?x:y) = 233

sizeof(int) = 4 bytes
sizeof(x)   = 4 bytes

== Casting desde x ==
char        = -71
short       = 185
int         = 185
long        = 185
long long   = 185
float       = 185.000000
double      = 185.000000
long double = 185.000000
```

operadorconlongint.c

Resultado:

```
===== TIPO: long int =====
x = 167
y = 74

== Aritmeticos (long int) ==
x + y  = 241  -> long: 241
x - y  = 93  -> long: 93
x * y  = 12358  -> long: 12358
x / y  = 2  -> long: 2 (division entera)
x % y  = 19  -> long: 19
+x     = 167
-x     = -167

== Incremento/Decremento ==
t inicial = 167
t++  imprime 167, luego t=168
++t  ahora t=169
t--  imprime 169, luego t=168
--t  ahora t=167

== Relacionales ==
x <  y  -> 0
x <= y  -> 0
x >  y  -> 1
x >= y  -> 1
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
(x && y)  -> 1
(x || y)  -> 1
```

```
== Bits ==
~x        = 4294967128
x & y     = 2
x | y     = 239
x ^ y     = 237
x << 1    = 334
x >> 1    = 83

== Asignacion ==
a = x = 167
a += y = 241
a -= y = 167
a *= y = 12358
a /= y = 167
a %= y = 19
a <<= 1 = 38
a >>= 1 = 19
a &= y = 2
a ^= y = 72
a |= y = 74

== Ternario y sizeof ==
mayor (x>y?x:y) = 167
sizeof(long int) = 4 bytes
sizeof(x)        = 4 bytes

== Casting desde x ==
char        = -89
short       = 167
int         = 167
long int    = 167
long long   = 167
float       = 167.000000
double      = 167.000000
long double = 167.000000
```

operadorconlong.c

Resultado:

```
===== TIPO: long =====
x = 47
y = 155

== Aritmeticos (long) ==
x + y  = 202  -> long: 202
x - y  = -108  -> long: -108
x * y  = 7285  -> long: 7285
x / y  = 0  -> long: 0 (division entera)
x % y  = 47  -> long: 47
+x     = 47
-x     = -47

== Incremento/Decremento ==
t inicial = 47
t++  imprime 47, luego t=48
++t  ahora t=49
t--  imprime 49, luego t=48
--t  ahora t=47

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
(x && y)  -> 1
(x || y)  -> 1
```

```
== Bits ==
~x        = 4294967248
x & y     = 11
x | y     = 191
x ^ y     = 180
x << 1    = 94
x >> 1    = 23

== Asignacion ==
a = x = 47
a += y = 202
a -= y = 47
a *= y = 7285
a /= y = 47
a %= y = 47
a <<= 1 = 94
a >>= 1 = 47
a &= y = 11
a ^= y = 144
a |= y = 155

== Ternario y sizeof ==
mayor (x>y?x:y) = 155
sizeof(long) = 4 bytes
sizeof(x)    = 4 bytes

== Casting desde x ==
char        = 47
short       = 47
int         = 47
long        = 47
long long   = 47
float       = 47.000000
double      = 47.000000
long double = 47.000000
```

operadorconlonglong.c

Resultado:

```
===== TIPO: long long =====
x = 48
y = 127

== Aritmeticos (long long) ==
x + y  = 175
x - y  = -79
x * y  = 6096
x / y  = 0 (division entera)
x % y  = 48
+x     = 48
-x     = -48

== Incremento/Decremento ==
t inicial = 48
t++  imprime 48, luego t=49
++t  ahora t=50
t--  imprime 50, luego t=49
--t  ahora t=48

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
(x && y)  -> 1
(x || y)  -> 1
```

```
== Bits ==
~x        = 18446744073709551567
x & y     = 48
x | y     = 127
x ^ y     = 79
x << 1    = 96
x >> 1    = 24

== Asignacion ==
a = x = 48
a += y = 175
a -= y = 48
a *= y = 6096
a /= y = 48
a %= y = 48
a <<= 1 = 96
a >>= 1 = 48
a &= y = 48
a ^= y = 79
a |= y = 127

== Ternario y sizeof ==
mayor (x>y?x:y) = 127
sizeof(long long) = 8 bytes
sizeof(x)         = 8 bytes

== Casting desde x ==
char        = 48
short       = 48
int         = 48
long        = 48
long long   = 48
float       = 48.000000
double      = 48.000000
long double = 48.000000
```

operadorconfloat.c

Resultado:

```
===== TIPO: float =====
x = 54.000000
y = 95.000000

== Aritmeticos (float) ==
x + y  = 149.0000000000  -> float: 149.000000
x - y  = -41.0000000000  -> float: -41.000000
x * y  = 5130.0000000000  -> float: 5130.000000
x / y  = 0.5684210526  -> float: 0.568421
fmodf(x, y) = 54.000000
+x     = 54.000000
-x     = -54.000000

== Incremento/Decremento ==
t inicial = 54.000000
t++  imprime 54.000000, luego t=54.000000
++t  ahora t=56.000000
t--  imprime 56.000000, luego t=56.000000
--t  ahora t=54.000000

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
(x && y)  -> 1
(x || y)  -> 1
```

```
== Bits ==
NO APLICA para float (solo enteros)

== Asignacion ==
a = x = 54.000000
a += y = 149.000000
a -= y = 54.000000
a *= y = 5130.000000
a /= y = 54.000000
NO APLICA: %, %, <<=, >>=, &=, ^=, |= para float

== Ternario y sizeof ==
mayor (x>y?x:y) = 95.000000
sizeof(float) = 4 bytes
sizeof(x)     = 4 bytes

== Casting desde x ==
char        = 54
short       = 54
int         = 54
long        = 54
long long   = 54
float       = 54.000000
double      = 54.000000
long double = 54.000000
```

¿Por qué no existen operadores de bits en `float`, `double` y `long double`?

Los operadores de bits (`&`, `|`, `^`, `~`, `<<`, `>>`) solo están definidos para tipos enteros en C.

Esto se debe a que los enteros representan valores binarios directos, donde cada bit forma parte del número completo.

En cambio, los tipos `float`, `double` y `long double` se almacenan usando el estándar IEEE-754, cuya representación interna está dividida en:

- 1 bit de signo
- Bits de exponente
- Bits de mantisa

Es decir, sus bits no representan un número entero continuo, sino una estructura especial para números en punto flotante.

Aplicar operaciones de bits alteraría esa estructura interna y produciría valores inválidos o indefinidos, por eso el lenguaje no permite estos operadores en tipos flotantes.

operadorcondouble.c

Resultado:

```
===== TIPO: double =====
x = 79.0000000000
y = 42.0000000000

== Aritmeticos (double) ==
x + y  = 121.00000000000000000000  -> double:
121.0000000000
x - y  = 37.00000000000000000000  -> double:
37.0000000000
x * y  = 3318.00000000000000000000  -> double:
3318.0000000000
x / y  = 1.88095238095238095238  -> double: 1.8809523810
fmod(x, y) = 37.0000000000
+x     = 79.0000000000
-x     = -79.0000000000

== Incremento/Decremento ==
t inicial = 79.0000000000
t++  imprime 79.0000000000, luego t=80.0000000000
++t  ahora t=81.0000000000
t--  imprime 81.0000000000, luego t=80.0000000000
--t  ahora t=79.0000000000

== Relacionales ==
x <  y  -> 0
x <= y  -> 0
x >  y  -> 1
x >= y  -> 1
x == y  -> 0
x != y  -> 1

== Logicos ==
```

```
!x         -> 0
(x && y)   -> 1
(x || y)   -> 1

== Bits ==
NO APLICA para double (solo enteros)

== Asignacion ==
a = x = 79.0000000000
a += y = 121.0000000000
a -= y = 79.0000000000
a *= y = 3318.0000000000
a /= y = 79.0000000000
NO APLICA: %, %, <<=, >>=, &=, ^=, |= para double

== Ternario y sizeof ==
mayor (x>y?x:y) = 79.0000000000
sizeof(double) = 8 bytes
sizeof(x)      = 8 bytes

== Casting desde x ==
char        = 79
short       = 79
int         = 79
long        = 79
long long   = 79
float       = 79.000000
double      = 79.0000000000
long double = 79.000000
```

operadorconlongdouble.c

Resultado:

```
===== TIPO: long double =====
x = 102.000000000000000
y = 221.000000000000000

== Aritmeticos (long double) ==
x + y  = 323.00000000000000000000
x - y  = -119.00000000000000000000
x * y  = 22542.00000000000000000000
x / y  = 0.46153846153846153847
fmodl(x, y) = 102.00000000000000000000
+x     = 102.00000000000000000000
-x     = -102.00000000000000000000

== Incremento/Decremento ==
t inicial = 102.000000000000000
t++  imprime 102.000000000000000, luego
t=103.000000000000000
++t  ahora t=104.000000000000000
t--  imprime 104.000000000000000, luego
t=103.000000000000000
--t  ahora t=102.000000000000000

== Relacionales ==
x <  y  -> 1
x <= y  -> 1
x >  y  -> 0
x >= y  -> 0
x == y  -> 0
x != y  -> 1

== Logicos ==
!x        -> 0
```

```
(x && y)  -> 1
(x || y)  -> 1

== Bits ==
NO APLICA para long double (solo enteros)

== Asignacion ==
a = x = 102.000000000000000
a += y = 323.000000000000000
a -= y = 102.000000000000000
a *= y = 22542.000000000000000
a /= y = 102.000000000000000
NO APLICA: %, <<=, >>=, &=, ^=, |= para long double

== Ternario y sizeof ==
mayor (x>y?x:y) = 221.000000000000000
sizeof(long double) = 16 bytes
sizeof(x)           = 16 bytes

== Casting desde x ==
char        = 102
short       = 102
int         = 102
long        = 102
long long   = 102
float       = 102.000000
double      = 102.000000
long double = 102.000000000000000
```

Conclusiones:

El poder conocer cómo se comportan los distintos tipos de datos a través de los diferentes operadores dentro del lenguaje C me permitió entender de una forma más clara cómo trabaja realmente el lenguaje a nivel interno. No solo se trata de realizar operaciones, sino de comprender qué ocurre en memoria dependiendo del tipo de dato utilizado.

Uno de los aspectos más importantes que observé en la práctica fue el desbordamiento de datos. El tipo `char`, al ser el más limitado en tamaño (8 bits), permite visualizar fácilmente este fenómeno. Por ejemplo, cuando el resultado de una operación supera el límite de 255 en un `unsigned char`, el valor no genera un error, sino que se ajusta automáticamente dentro del rango permitido. En el caso de 279, como solo se pueden representar valores entre 0 y 255, el número “reinicia” y continúa desde 0 hasta llegar a 23. Esto me permitió entender que los tipos sin signo trabajan con aritmética modular y que el desbordamiento es un comportamiento definido, no un error del programa.

También aprendí cómo funcionan operadores que son muy característicos del lenguaje C, como los operadores de bits. Comprendí por qué solo pueden aplicarse a números enteros y no a números decimales, ya que los tipos flotantes manejan una representación diferente en memoria (signo, exponente y mantisa). Esto me ayudó a diferenciar claramente entre la aritmética entera y la aritmética de punto flotante.

En general, esta práctica me permitió entender que en C el tipo de dato influye directamente en el resultado de las operaciones, en el uso de los operadores y en el comportamiento del programa. No todos los tipos se comportan igual, y conocer sus límites es fundamental para evitar errores y comprender realmente lo que está ocurriendo a nivel binario.

Referencias

Chat GPT | Modelo 5.2
https://learn.microsoft.com/es-es/cpp/c-language/c-bitwise-operators?view=msvc-170
w3schools